

1. 当前思路的可行性评价

你的方向是可行的，而且是比较清晰的研究切入点。多智能体辩论本身已有相关工作支持：多个语言模型实例提出答案并多轮辩论，能够在若干推理任务上提升表现，并且这种方法可以直接用于黑盒模型。(arXiv) 你的方案进一步把通信结构从全连接辩论压缩成环形辩论，复杂度从近似 $O(N^2 \times R)$ 降到 $O(N \times R)$ ，这对医学 MCQA 这种需要大量题目评测的任务很重要。

你当前的主要创新点不应只写成“多智能体辩论用于医学选择题”，而应写成：

环形局部辩论 + 选项级上下文压缩 + 稳定性驱动早退。

这样贡献更明确，也更容易做消融实验。

不过，当前方案有三个风险：

第一，只按“答案相同”压缩理由，在多选题上会丢信息。例如答案都是 {A, C}，但一个 agent 是因为支持 A、C，另一个是因为排除 B、D；这两种解释对后续辩论的价值不同。如果直接合并为“选择 A,C 的理由”，会丢掉“为什么不选 B/D”的反证信息。

第二，“两轮答案不变就早退”可能过早停止在错误共识上。医学选择题的准确率本身不一定代表可靠性，ReMedQA 最近专门指出，医学 MCQA 高准确率可能掩盖对选项格式、扰动和表述变化的敏感性，并提出用一致性指标评估可靠性。(disi-unibo-nlp.github.io) 所以早退机制最好不仅看“是否不变”，还要看“是否稳定且争议选项已经消失”。

第三，MedMCQA 的多选标签要先做数据核验。官方说明中，MedMCQA 任务可以选择单个或多个答案，并且字段包含 choice_type 与专家解释 exp；但字段 cop 又被描述为 correct option，因此你应先统计 choice_type=multi-choice 的样本是否真的存在多正确标签，再决定是否把它作为真正多标签数据集使用。(GitHub)

2. 多选题不要看成“答案字符串”，而要看成“选项级二分类”

对单选题，答案空间是 {A,B,C,D}；对多选题，答案空间是 $\{0,1\}^K$ ，例如 4 个选项时共有 16 种组合。因此多选题里最重要的改变是：

不再压缩“最终答案集合”，而是压缩“每个选项是否应被选择的理由”。

设第 i 个 agent 在第 t 轮的答案为：

$[x_{i,t} \in \{0,1\}^K]$

其中 $x_{i,k} = 1$ 表示 agent i 认为第 k 个选项应被选中。全局投票为：

$[p_{k,t} = \frac{1}{N} \sum_i x_{i,k}^t]$

最终答案可以用阈值决定：

$[\hat{x}_k = 1[p_{k,t} \geq \tau]]$

推荐初始设置：

- 单选题：强制只选 $\arg\max_k p_k$
- 多选题：默认 $\tau = 0.5$
- 如果验证集可用：在 dev set 上搜索 $\tau \in \{0.4, 0.5, 0.6\}$ ，以 exact match 或 macro-F1 为准

3. 建议的 agent 输出格式

为了实现压缩和早退，所有 agent 都必须输出结构化结果。不要让模型自由长篇推理，而是输出“短理由 + 选项级判断”。

```
{
  "selected_options": ["A", "D"],
  "per_option": {
    "A": {
      "verdict": "include",
      "confidence": 0.82,
      "reason": "A 与题干中的关键症状/机制一致。"
    },
    "B": {
      "verdict": "exclude",
      "confidence": 0.76,
      "reason": "B 缺少题干所需的核心条件。"
    },
    "C": {
      "verdict": "exclude",
      "confidence": 0.61,
      "reason": "C 可解释部分信息，但与某个限制条件冲突。"
    },
    "D": {
      "verdict": "include",
      "confidence": 0.74,
      "reason": "D 与诊断/治疗原则一致。"
    }
  },
  "changed_from_previous": ["C"],
  "critique_to_previous_agent": "上一位 agent 选择 C 的依据不足，因为题干中没有支持 C 的必要条件。"
}
```

这样你可以直接解析：

- 当前答案集合
- 每个选项的 include / exclude 立场
- 每个选项的置信度
- 本轮发生变化的选项
- 对上一个 agent 的局部反驳

这比自然语言辩论容易实现很多。

4. 多选题上下文压缩：从“答案桶”改成“选项-立场账本”

你当前的压缩思路是“答案相同的理由进行总结”。单选题可以这么做，但多选题建议升级为：

option-level stance ledger，即“选项-立场账本”。

每个选项维护两组理由：

```
Option A:
  include:
    - 支持理由 1，支持 agents: [1,3,5]，平均置信度 0.81
    - 支持理由 2，支持 agents: [2]
  exclude:
    - 反对理由 1，支持 agents: [4]，平均置信度 0.68

Option B:
  include:
    - ...
  exclude:
    - ...
```

压缩时不要保留所有理由，只保留每个选项每个方向的 Top-M 条理由。推荐最小实现：

- 每个选项最多保留 2 条 include 理由
- 每个选项最多保留 2 条 exclude 理由
- 每条理由保留：summary, support_count, agent_ids, avg_confidence
- 对稳定选项只保留一句摘要
- 对不稳定选项保留正反两方理由

例如 4 个选项时，最多是：

$[4 \times \text{options} \times 2 \times \text{stances} \times 2 \times \text{claims} = 16]$

也就是最多 16 条短理由，token 成本很可控。

压缩规则建议

最容易实现的是 LLM summarizer 压缩，不需要训练模型。

每次新 agent 输出后，把旧 ledger 和新理由交给一个便宜模型或同一个模型的低温调用：

```
请更新以下选项级辩论账本。
要求：
1. 合并语义重复的理由。
2. 不要合并 include 与 exclude 的相反理由。
3. 每个选项每个方向最多保留 2 条理由。
4. 保留少数派但关键的反驳。
5. 输出 JSON。
```

理由优先级可以按这个简单打分：

$[score = support_count + 0.5 \times avg_confidence + conflict_bonus]$

其中 `conflict_bonus = 1` 表示该理由针对的是当前争议选项。

5. 环形辩论中的“压缩胶囊”

你希望每个 agent 只对上一个 agent 的答案和解释辩论。可以保持这个设定，但让上一个 agent 传递一个压缩后的 debate capsule，而不是传递全部历史。

每个 agent 看到：

```
1. 原始问题和选项
2. 自己上一轮答案
3. 上一个 agent 的答案、解释和反驳
4. 压缩后的选项级账本
5. 当前未稳定的选项列表
```

示例：

```
[Compressed Debate Capsule]

Global votes:
A: include 4/5, stable
B: include 2/5, unstable
C: include 0/5, stable
D: include 3/5, unstable

Stable options:
A is likely included.
C is likely excluded.

Unstable options to discuss:
B, D

Key claims:
B include:
- B matches symptom X. agents=[2,4], avg_conf=0.71
B exclude:
- B conflicts with condition Y in the stem. agents=[1,3,5], avg_conf=0.78

D include:
- D is supported by mechanism Z. agents=[1,2,5], avg_conf=0.76
D exclude:
- D may be contraindicated under condition W. agents=[3], avg_conf=0.69

Previous agent answer:
Selected: B, D
Critique: ...
```

这样形式上仍然是“只看上一个 agent 的输出”，但上一个 agent 携带了压缩状态。实现上也很简单：每个节点更新 ledger 后传给下一个节点。

6. 多选题早退机制：建议用“三层早退”

你现在的早退机制是：

走两轮后，所有 agent 答案都没有变化，则提前结束。

这个可以保留为 **hard early exit**，但多选题建议增加两个更细粒度机制。

6.1 Hard early exit：全体 agent 答案向量不变

对每个 agent 的完整多选向量做比较：

$$[x_i^t = x_i^{t-1}, \forall \text{all } i]$$

如果连续 $L=2$ 个完整环都满足，停止。

优点：安全、简单。缺点：太保守，一个小选项反复变化会拖住整个问题。

6.2 Option-level early exit：按选项冻结

对每个选项单独判断是否稳定。

选项 k 满足以下任一条件即可冻结：

- 所有 agent 都同意选它或都不同意不选它： $[\text{vote}_k \in \{0, N\}]$
- 连续 $L=2$ 个环，投票数不变，并且远离阈值： $[|p_k - \tau| \geq m]$

推荐初始值：

- $\tau = 0.5$
- $m = 0.2$
- $L = 2$

例如 $N=5$ 时：

- A: 5/5 include，直接冻结

- B: 4/5 include，若两轮不变，冻结为 include
- C: 1/5 include，若两轮不变，冻结为 exclude
- D: 2/5 或 3/5，继续辩论

冻结后的选项不再进入后续 prompt，只在 capsule 中写一句：

```
A frozen as include, vote=5/5.
C frozen as exclude, vote=0/5.
```

这会显著减少 token。

6.3 Deadlock early exit：争议模式重复则停止

多选题常见情况是某个选项一直 2 vs 3 或 3 vs 2。这时继续辩论可能只是重复。

定义全局最终答案：

$\hat{x}^t = 1[p_k^t \geq \tau]$

如果连续 $L_{\text{deadlock}} = 3$ 个环满足：

```
global answer unchanged
AND unstable option vote pattern unchanged
AND no new claim added to ledger
```

则停止，进入最终聚合。

最终聚合建议：

1. 没有争议：直接 majority vote
2. 有争议：使用 confidence-weighted vote
3. 仍然平票：只对争议选项调用一个 judge，而不是让 judge 重看全题

confidence-weighted vote：

$score_k = \frac{1}{N} \sum_i c_{i,k} \cdot (2x_{i,k} - 1)$

如果 $score_k > 0$ ，则选入；否则不选。

7. 最小可实现算法

```
for question in dataset:
    initialize agents independently
    initialize option_ledger
    frozen = {option: False for option in options}
    unchanged_full_cycles = 0
    unchanged_global_cycles = 0

    for cycle in range(max_cycles):
        changed_any = False
        new_claim_added = False

        for i in range(N):
            prev = agents[(i - 1) % N]
            cur = agents[i]

            context = build_context(
                question=question,
                options=options,
                previous_agent_answer=prev.answer,
                previous_agent_explanation=prev.explanation,
                compressed_ledger=option_ledger,
                frozen_options=frozen
            )

            new_output = call_agent(cur, context)
            parsed = parse_json(new_output)
```

```

        if parsed.answer_vector != cur.answer_vector:
            changed_any = True

        cur.update(parsed)

        option_ledger, added = update_ledger(option_ledger, parsed)
        new_claim_added = new_claim_added or added

    vote_vector = compute_votes(agents)
    final_answer = threshold_votes(vote_vector, tau=0.5)

    frozen = update_frozen_options(
        vote_vector=vote_vector,
        history=vote_history,
        tau=0.5,
        margin=0.2,
        patience=2
    )

    if not changed_any:
        unchanged_full_cycles += 1
    else:
        unchanged_full_cycles = 0

    if final_answer == previous_final_answer:
        unchanged_global_cycles += 1
    else:
        unchanged_global_cycles = 0

    if unchanged_full_cycles >= 2:
        break

    if all(frozen.values()):
        break

    if unchanged_global_cycles >= 3 and not new_claim_added:
        break

return aggregate_final_answer(agents, option_ledger)

```

8. 实验设计建议

必做 baseline

Baseline	作用
Single Agent	判断多智能体是否真的有增益
Self-Consistency / Majority Vote	判断“多样采样 + 投票”是否已经足够；self-consistency 的核心是采样多条推理路径并选择最一致答案。(arXiv)
Ring Debate without Compression	验证压缩是否影响准确率
Ring Debate with Answer-Bucket Compression	对比你原始压缩方案
Ring Debate with Option-Ledger Compression	你的主方法
Fixed-Round Debate	对比早退是否节省成本
Early-Exit Debate	你的完整方法

指标

单选题：

- Accuracy
- 平均调用次数
- 平均 token 数
- 答案变化轮数
- 早退比例

多选题：

- Exact Match
- Micro-F1
- Macro-F1
- Jaccard Similarity
- Hamming Loss
- Precision / Recall
- 每题平均调用次数
- 每题平均 token 数
- 每个选项的稳定轮数

医学选择题建议同时看 reliability，不要只看 accuracy。ReMedQA 的思路可以借鉴：对同一问题做格式扰动或选项扰动，观察答案是否一致。[\(disi-unibo-nlp.github.io\)](#)

9. Agent 设计：先简单，不要一开始复杂化

推荐从 N=3 或 N=5 开始。

最简单配置：

```
Agent 1: general medical reasoning
Agent 2: diagnosis / pathophysiology focused
Agent 3: treatment / pharmacology focused
Agent 4: skeptical reviewer
Agent 5: evidence and option-elimination focused
```

每个 agent 的 prompt 不要差异太大，否则难以判断提升来自“辩论”还是“角色 prompt”。建议先做两个版本：

1. **Homogeneous agents**：同模型、同 prompt、不同采样 seed
2. **Light heterogeneous agents**：同模型、不同关注点 prompt

如果 heterogeneous 明显更好，再进一步分析角色贡献。

10. 更合适的数据集

你的 MedQA + MedMCQA 组合是合理起点。MedQA 来自医学专业考试，覆盖英文、简体中文、繁体中文三个版本，分别包含 12,723、34,251、14,123 个问题。[\(arXiv\)](#) MedMCQA 的优势是有 choice_type、专家解释、学科和主题字段，适合做解释压缩和分科分析。[\(GitHub\)](#)

后续可以按研究目的补充这些数据集：

数据集	适合用途	说明
CMB	中文、多正确答案、多专业层级	CMB-Exam 包含资格考试选择题，CMB-Clin 包含复杂病例咨询；选择题有 4–6 个选项，且可以有一个或多个正确答案，特别适合真正多选题实验。
CMEExam	中文医学考试 + 解释评估	来源于中国国家医学资格考试，包含 60K+ 选择题、解题解释，以及疾病组、科室、学科、能力领域、难度等标注。 (arXiv)
MediQAI	法语，多选/单选/开放题	包含 32,603 个法国医学考试问题，其中有唯一答案 MCQ、多答案 MCQ 和短开放题，适合验证多选机制的跨语言泛化。 (Nature)
FrenchMedMCQA	小规模真实多选	由法国药学科考试试题构成，包含 3,105 个问题，混合单答案和多答案，每题有 5 个选项和人工订正。 (ACL Anthology)
MMedBench	多语言选择题 + rationale	面向 6 种语言的医学多选基准，带 rationale；适合测试你的压缩机制是否依赖英文。 (Nature)
MedExpQA / MedExQA	解释质量评估	MedExQA 强调每个问答对有多个解释，适合评估“压缩后的理由是否仍能支持正确答案”。 (arXiv)
ReMedQA	鲁棒性和早退可靠性	它扩展 MedQA、MedMCQA 和 MMLU medical subsets，构造开放答案和系统扰动版本，适合测试早退是否会在扰动下变得不可靠。 (disi-unibo-nlp.github.io)

数据集	适用用途	说明
PubMedQA	biomedical reasoning	任务是基于 PubMed 摘要回答 yes/no/maybe，包含专家标注、未标注和自动生成样本；不是真正多选，但适合测试“辩论是否能利用证据”。(ACL Anthology)
BioASQ-QA	专家级生物医学 QA / 检索增强	包含英文问题、参考答案和相关材料，反映生物医学专家的信息需求，适合做 RAG + debate。(Nature)
LiveQA Medical Task	消费者健康问答	面向美国国家医学图书馆收到的消费者健康问题，适合在 MCQA 方法稳定后扩展到开放式医学问答。(lhncbc.nlm.nih.gov)

优先级建议：

1. 第一阶段：MedQA + MedMCQA
2. 第二阶段：CMB 或 MediQAI，用来验证真正多正确答案
3. 第三阶段：ReMedQA，用来测试早退机制的鲁棒性
4. 第四阶段：MedExQA / BioASQ，用来测试解释压缩是否有医学意义

11. 可以写成论文方法的核心版本

你的方法可以命名为类似：

Ring-MAD with Option-Level Memory and Stability Early Exit

核心模块：

1. Independent Initialization 每个 agent 独立回答问题，输出选项级 JSON。
2. Ring Debate agent 只接收上一个 agent 的答案、解释和压缩胶囊。
3. Option-Level Memory Compression 对每个选项维护 include / exclude 理由账本，而不是按最终答案集合压缩。
4. Unstable-Only Debate 后续只辩论未冻结选项，稳定选项不再展开讨论。
5. Stability Early Exit 使用 hard stability、option-level stability 和 deadlock stability 三层早退。
6. Final Aggregation 单选题用 argmax；多选题用 threshold vote；争议选项用 confidence-weighted vote 或轻量 judge。

12. 最推荐的下一步

先实现一个最小实验：

- N = 5
- max_cycles = 4
- 每个选项最多保留 2 条 include 理由和 2 条 exclude 理由
- 早退阈值：连续 2 个环无变化；或所有选项冻结；或连续 3 个环 deadlock
- 数据集先用 MedQA + MedMCQA validation
- 多选评价用 Exact Match + Macro-F1 + Jaccard
- 成本评价用 average calls + average tokens

这个版本足够形成一个清晰的主实验。之后再加 CMB / MediQAI 做真正多选验证，会比一开始就堆很多数据集更稳。